# MAC — MBM AI Cloud

*A Self-Hosted AI Inference Platform for Campus-Wide Deployment*

Final Project Report Submitted in Partial Fulfilment
of the Requirements for the Degree of

## Bachelor of Engineering

*in*

## Computer Science & Engineering


*Submitted by*

Aaryan Choudhary : (Roll No. 23UCSE4002)

*Under the Supervision of*

Dr. Abhisek Gour
Professor, Computer Science & Engineering
MBM University, Jodhpur




**Department of Computer Science and Engineering**
MBM University, Jodhpur


**May, 2026**

**Department of Computer Science & Engineering**
MBM University, Ratanada, Jodhpur, Rajasthan, India – 342011

# CERTIFICATE

This is to certify that the work contained in this report entitled **"MAC — MBM AI Cloud: A Self-Hosted AI Inference Platform for Campus-Wide Deployment"** is submitted by Mr. **Aaryan Choudhary** (Roll No. 23UCSE4002) to the Department of Computer Science & Engineering, MBM University, Jodhpur, for the partial fulfilment of the requirements for the degree of **Bachelor of Engineering** in **Computer Science & Engineering**.

He has carried out his work under my supervision. This work has not been submitted elsewhere for the award of any other degree or diploma.

The project work in our opinion has reached the standard fulfilling the requirements for the degree of Bachelor of Engineering in accordance with the regulations of the University.

**Dr. Abhisek Gour**
Professor, CSE
(Supervisor)

**Dr. N. C. Barwar**
(Head of Department)

# DECLARATION

I, ***Aaryan Choudhary***, hereby declare that this project report titled **"MAC — MBM AI Cloud: A Self-Hosted AI Inference Platform for Campus-Wide Deployment"** is a record of original work done by me under the supervision and guidance of ***Dr. Abhisek Gour***.

I further certify that this work has not formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or similar recognition at any university, and no part of this report is reproduced as it is from any other source without appropriate reference and permission.

SIGNATURE OF STUDENT

**(Aaryan Choudhary)**
8th Semester, Computer Science & Engineering
Roll No. – 23UCSE4002

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, **Dr. Abhisek Gour**, Professor, Department of Computer Science & Engineering, MBM University, Jodhpur, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the preparation of this report.

I am deeply thankful to **Dr. N. C. Barwar**, Professor & Head, Department of Computer Science & Engineering, for providing the necessary resources and an academically stimulating environment that enabled me to undertake this project.

I also acknowledge the open-source community and the authors of the tools and frameworks discussed herein — vLLM, FastAPI, PostgreSQL, Qdrant, Docker, and many more — whose published documentation, tutorials, and research papers formed the technical foundation of this work.

# ABSTRACT

**MAC (MBM AI Cloud)** is a fully self-hosted, on-premise AI inference platform designed and built for MBM University, Jodhpur. It provides every student and faculty member with access to large language models, document-aware AI chat, VS Code-style cloud notebooks (MBM Book IDE), voice interaction, and academic management tools — all running on commodity GPU hardware inside the campus LAN, with no internet dependency and no recurring cost.

The platform integrates a FastAPI asynchronous backend, PostgreSQL for relational data, Redis for token caching, Qdrant for vector similarity search (RAG), vLLM for high-throughput LLM inference (PagedAttention), Whisper for speech-to-text, and Veena TTS for text-to-speech. The frontend is a vanilla JavaScript SPA with Progressive Web App (PWA) support, offline capability, and 19-language internationalisation.

Key technical contributions include: a 3-PC GPU cluster with LAN-based node discovery and load balancing, JWT authentication with Redis token blacklisting, RAG pipeline with HNSW vector search, and per-user Docker workspace containers for the MBM Book IDE. The entire stack is packaged as a single `docker compose up` command.

**Keywords:** Self-hosted AI, LLM inference, Campus AI cloud, vLLM, Retrieval-Augmented Generation, FastAPI, Docker, Progressive Web App, MBM University.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Background and Motivation

India graduates over 1.5 million engineering students annually, yet access to modern AI tools remains concentrated in well-funded institutions. Premium commercial AI services cost between Rs. 1,000 and Rs. 5,000 per student per month — an amount that places them out of reach for the majority of state universities.

MBM University, Jodhpur, like most engineering colleges, possesses significant computing hardware in its laboratories: **multiple desktop PCs each equipped with an NVIDIA RTX 3060 GPU (12 GB VRAM)**. These machines sit idle outside class hours. The central question motivating this project is: *can these machines be transformed into a shared AI cloud that serves every student on campus, at no recurring cost, with no internet dependency?*

## 1.2 Problem Statement

Students at MBM University face three compounding challenges in adopting AI tools:

1. **Cost:** Cloud-based AI APIs charge per token. A single active student can consume Rs. 500–Rs. 2,000 per month.
2. **Connectivity:** Campus internet is intermittent. Cloud services fail when connectivity drops.
3. **Privacy:** Examination questions, personal academic data, and institutional knowledge must not leave campus boundaries.

No existing open-source platform addressed all three concerns simultaneously in a package deployable on commodity hardware by a student team.

## 1.3 Objectives

The project sets the following measurable objectives:

- **Deploy a fully offline AI chat interface** serving 1,000 concurrent students over campus WiFi.
- **Achieve time-to-first-token under 200 ms** for LLM inference on commodity RTX 3060 hardware.

- Implement Retrieval-Augmented Generation (RAG) so that students can attach course material and receive contextually accurate answers.
- Build academic tools — Q&A forum, computational notebooks — tightly integrated with the AI backend.
- Support 19 Indian and regional languages entirely offline.
- Design a horizontal scaling architecture allowing additional GPU PCs to join the cluster over the campus LAN.
- Achieve Progressive Web App (PWA) compliance for mobile and desktop installation.

## 1.4 Scope

This project covers the complete full-stack design and implementation of the MAC platform: database schema, asynchronous REST backend, AI/ML integration, frontend SPA, and Docker-based deployment automation.

**In scope:** AI chat, RAG, Jupyter-compatible notebooks, doubts forum, file sharing, voice chat, admin panel, feature flags, quota management, LAN node discovery, and multi-language UI.

**Out of scope:** Training or fine-tuning of language models. Only inference and integration of publicly available open-weight models is performed.

**Current status:** All core features listed above are fully operational. A Windows one-click installer (`.exe`) is under active development and currently being stabilised. Docker images are published to the GitHub Container Registry (GHCR).

## 1.5 System Use Case Overview

Figure 1.1 shows the three primary actors and their interactions with the MAC system.

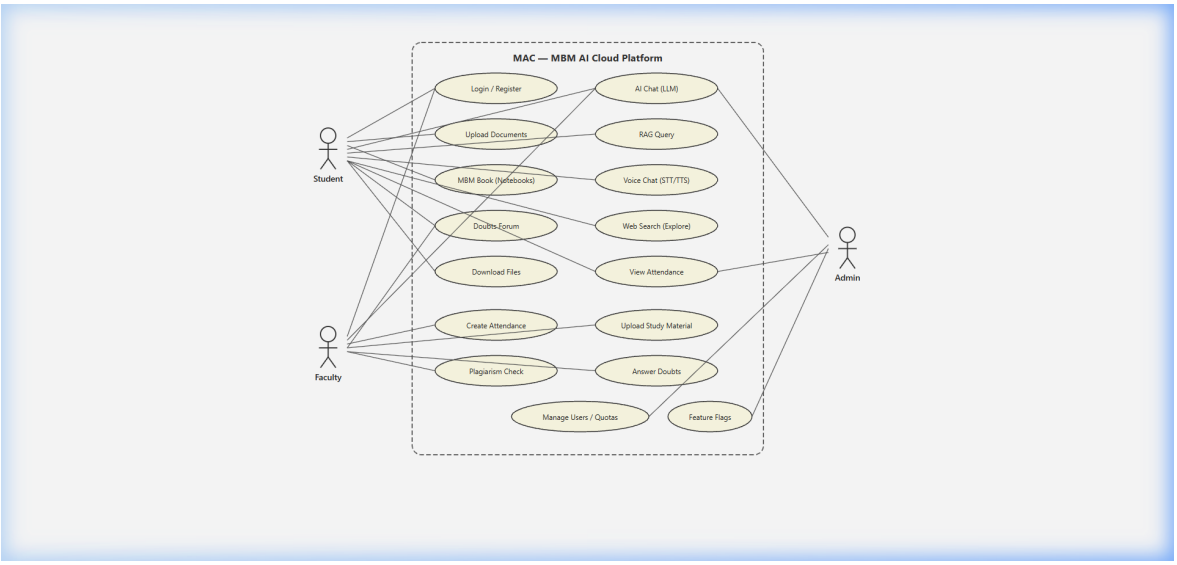


Figure 1.1: Use Case Diagram — MAC System Actors and Features

## 1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 surveys related literature and existing platforms. Chapter 3 describes the overall system architecture. Chapter 4 covers authentication and security in detail, including the mathematical foundations of JWT. Chapter 5 presents the AI and machine learning layer, including transformer attention, cosine similarity for vector search, and the RAG pipeline. Chapter 6 details the database design. Chapter 7 describes the frontend architecture. Chapter 8 documents all features and modules. Chapter 9 traces the development journey and SDLC methodology. Chapter 10 covers deployment and distribution. Chapters 11 and 12 present testing and limitations, and Chapter 13 concludes.

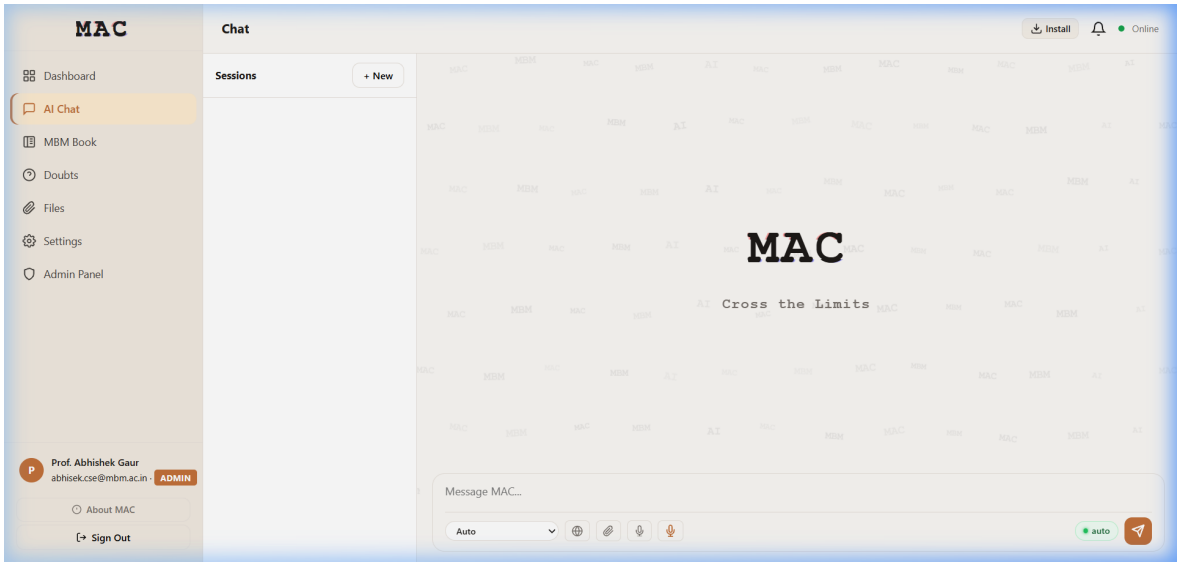


Figure 1.2: MAC AI Chat Interface — Main Landing Screen

# Chapter 2

# Literature Review and Related Work

## 2.1 Large Language Models for Education

Large language models (LLMs) — neural networks trained on massive corpora using the transformer architecture [1] — have demonstrated strong performance on question answering, code generation, and tutoring tasks. Studies such as [2] find that students who use LLM-based tools complete programming assignments 30% faster on average, but institutional deployment remains rare due to cost and privacy barriers.

## 2.2 vLLM and Efficient Inference

Serving LLMs efficiently at scale requires careful memory management. vLLM [3] introduces *PagedAttention*, a technique that partitions the KV-cache into fixed-size pages, dramatically reducing memory fragmentation and enabling higher throughput. On a single NVIDIA RTX 3060 (12 GB VRAM), vLLM can serve quantised 7B-parameter models at over 1,000 tokens per second — sufficient for a multi-user campus deployment.

## 2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [4] addresses the knowledge cutoff limitation of LLMs by retrieving relevant document passages at inference time and injecting them into the prompt. This allows accurate answers over course-specific material (lecture notes, past papers) without retraining or fine-tuning the model.

## 2.4 Distributed GPU Computing on Commodity Hardware

Research in federated inference [5] has shown that distributing transformer layers across consumer-grade GPUs connected via Gigabit Ethernet is feasible for academic clusters. MAC adopts a simpler request-routing model (routing full requests to available nodes) rather than layer-splitting, which avoids inter-GPU synchronisation overhead.

# Chapter 3

# System Architecture

## 3.1 Architectural Overview

MAC follows a layered, service-oriented architecture deployed entirely via Docker Compose on a campus LAN. Figure 3.1 shows the four primary layers and the components within each.

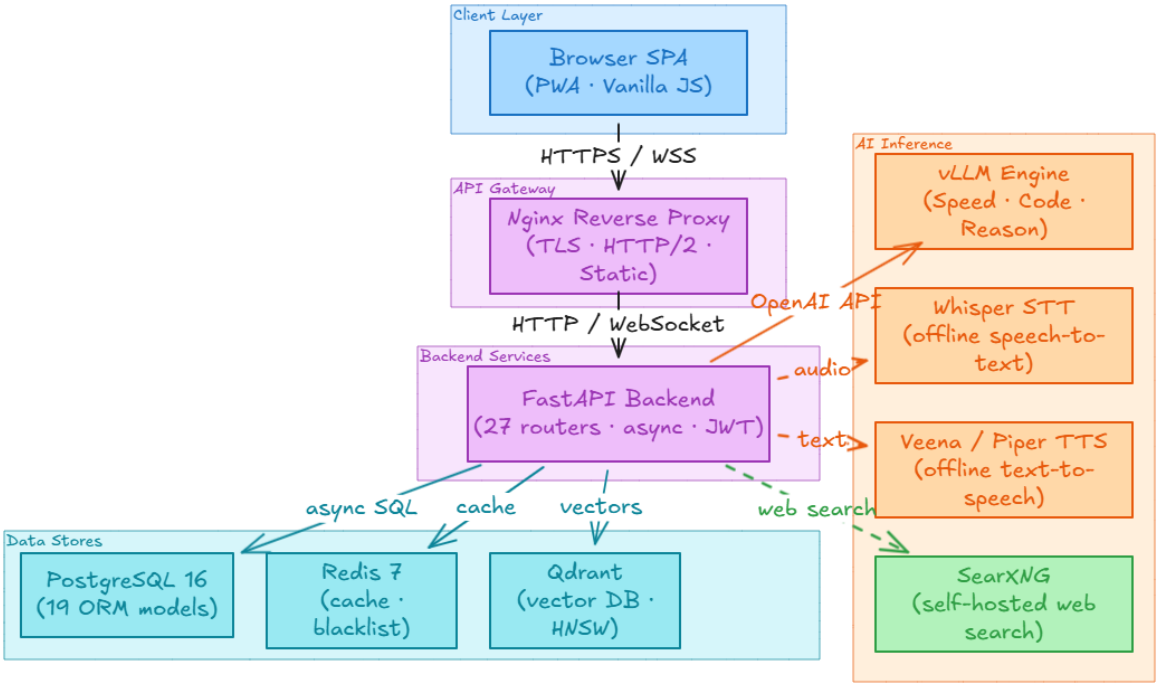


Figure 3.1: MAC System Architecture

Table 3.1: Architectural Layers and Responsibilities

| **Layer** | **Responsibility** |
|---|---|
| Client Layer | Browser SPA (Vanilla JS + PWA), mobile installs, Windows Worker Agent |
| API Gateway | Nginx reverse proxy — TLS termination, rate limiting, static file serving |
| Backend Layer | FastAPI application — routing, auth, business logic, LLM orchestration |
| Data Layer | PostgreSQL (relational), Redis (cache/blacklist), Qdrant (vector store) |
| AI Cluster | vLLM inference nodes (GPU), Veena TTS, Whisper STT — LAN-connected PCs |

## 3.2 Request Lifecycle

Every user request passes through the following pipeline:

1. The client browser sends an HTTPS request to **Nginx** on port 443.
2. Nginx terminates TLS, applies rate-limiting rules, and proxies to **FastAPI** on port 8000.
3. FastAPI's `AuthMiddleware` validates the JWT from the `Authorization` header and checks the token against the Redis blacklist.
4. The appropriate router handler is invoked. For AI requests, the **LLM Service** selects the best available vLLM node via the load balancer.
5. The response (plain JSON or a streaming `text/event-stream`) flows back through Nginx to the browser.

## 3.3 Data Flow Diagram

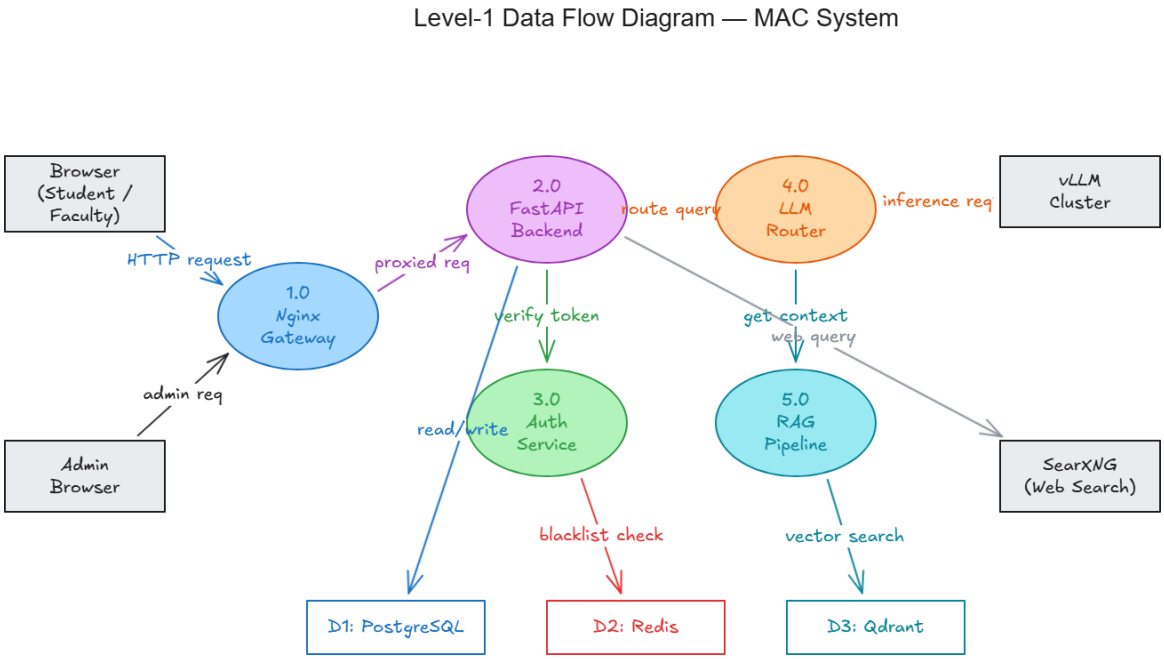


Figure 3.2: Level-1 Data Flow Diagram — MAC system

The **3-PC cluster** used in production consists of:

- **PC-1 (Main Server):** Runs the full Docker Compose stack — FastAPI, PostgreSQL, Redis, Qdrant, Nginx, SearXNG, and vLLM inference (RTX 3060, 12 GB VRAM).
- **PC-2 (Worker Node):** Optional additional inference capacity. Registers with PC-1 via LAN discovery (UDP broadcast on port 5005).
- **PC-3 (Voice Node):** Dedicated to speech processing — Whisper STT and Veena TTS. Kept separate because TTS/STT are CPU-intensive and would compete with GPU inference.

# Chapter 4

# Authentication and Security

## 4.1 JSON Web Tokens — Mathematical Foundation

Authentication in MAC uses **JSON Web Tokens (JWT)** [6] with the HMAC-SHA-256 signing algorithm. A JWT consists of three Base64URL-encoded parts separated by dots:

$$\text{JWT} = \underbrace{\text{Base64URL}(H)}_{\text{Header}} \,.\, \underbrace{\text{Base64URL}(P)}_{\text{Payload}} \,.\, \underbrace{\text{Base64URL}(S)}_{\text{Signature}}$$

The **signature** is computed as:

$$S = \text{HMAC-SHA256}(K,\ \text{Base64URL}(H) \,\|\, . \,\|\, \text{Base64URL}(P))$$

where $K$ is the server's secret key (stored in `MAC_SECRET_KEY`) and $\|$ denotes string concatenation. The server verifies a token by recomputing $S$ and comparing it to the received value. Because HMAC is a keyed function, an attacker without $K$ cannot forge a valid signature.

**Token expiry.** Every token carries an `exp` claim (Unix timestamp). The server rejects tokens where now() > exp. Short-lived access tokens (30 minutes) are paired with longer-lived refresh tokens (7 days).

**Token blacklisting.** On logout, the token's unique `jti` (JWT ID) is written to Redis with TTL equal to the token's remaining lifetime:

$$\texttt{SETEX}\ \text{blacklist:}\langle jti \rangle\ \Delta t\ \ 1 \quad \text{where } \Delta t = \text{exp} - \text{now}()$$

This ensures immediate invalidation without requiring database lookups on every request.

## 4.2 Password Hashing

User passwords are never stored in plaintext. The **bcrypt** algorithm is applied:

$$h = \text{bcrypt}(p,\ \text{cost} = 12)$$

bcrypt incorporates a random 128-bit **salt** into the hash, making precomputed rainbow-table attacks infeasible. A cost factor of 12 requires approximately 250 ms per hash on modern hardware — slow enough to deter brute force, fast enough for legitimate logins.

## 4.3 Role-Based Access Control (RBAC)

Three roles are defined: `admin`, `faculty`, and `student`. Each API endpoint is decorated with a dependency that enforces the minimum required role:

$$\text{Access granted} \iff \text{role}(u) \in \mathcal{R}_{\text{endpoint}}$$

where $\mathcal{R}_{\text{endpoint}}$ is the set of roles permitted for that endpoint. Role elevation (e.g., a faculty member seeing student data) is checked at the service layer, not just at the router.

## 4.4 Scoped API Keys

Beyond JWT, MAC supports long-lived **scoped API keys** for programmatic access (e.g., the Worker Agent). Each key is a 32-byte cryptographically random token, stored as its SHA-256 hash. The key carries a `scopes` bitmask:

$$\text{Scope} = \bigoplus_{i} 2^{s_i} \quad s_i \in \{\texttt{chat}, \texttt{read}, \texttt{write}, \texttt{admin}\}$$

On each request, the presented key is hashed and compared against stored hashes in $O(1)$ via a Redis lookup.

## 4.5 JWT Authentication — Sequence Diagram

Figure 4.1 shows the full token lifecycle from login through a protected API call to logout.

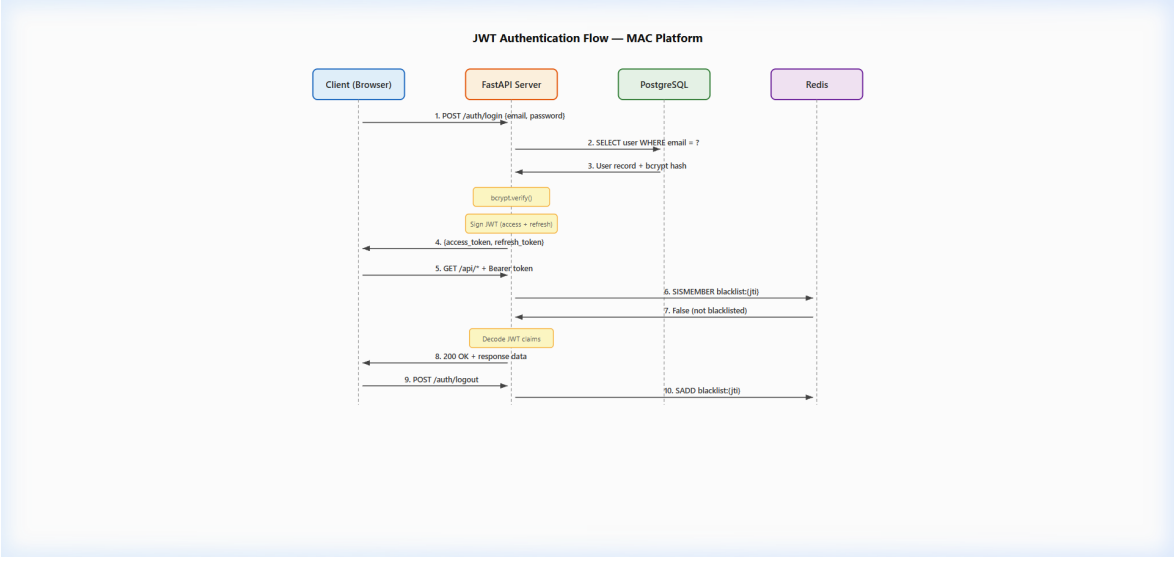


Figure 4.1: JWT Authentication — UML Sequence Diagram

## 4.6 AI and Machine Learning Layer

### 4.6.1 Transformer Architecture and Attention

All LLMs used by MAC are based on the **Transformer** architecture [1]. The core operation is **scaled dot-product attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where $Q \in \mathbb{R}^{n \times d_k}$ is the query matrix, $K \in \mathbb{R}^{m \times d_k}$ the key matrix, $V \in \mathbb{R}^{m \times d_v}$ the value matrix, and $d_k$ the key dimension. The $\sqrt{d_k}$ scaling factor prevents the dot products from growing too large in magnitude, which would push softmax into regions of vanishing gradient.

**Multi-head attention** runs $h$ attention heads in parallel and concatenates:

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V)\end{aligned}$$

### 4.6.2 vLLM and PagedAttention

Standard LLM serving pre-allocates GPU memory for the maximum sequence length, wasting up to 60–80% of KV-cache memory [3]. vLLM solves this with *PagedAttention*: the KV-cache is divided into fixed-size **pages** (analogous to virtual memory in an OS), allocated on demand:

$$\text{Memory used} = \sum_{\text{seq } s} \left\lceil \frac{|s|}{P} \right\rceil \cdot P \cdot d_{\text{model}} \cdot 2L \cdot 2 \text{ bytes}$$

where $P$ is page size, $L$ is the number of transformer layers, and the factor of 2 accounts for keys and values. On an RTX 3060 (12 GB), MAC serves a quantised Qwen2.5-7B (AWQ, 4-bit) at approximately 1,200 tokens/second.

### 4.6.3 Retrieval-Augmented Generation (RAG)

RAG [4] augments LLM responses with evidence retrieved from a local document corpus. The pipeline has two phases:

**Indexing (offline):**

1. Parse uploaded files (PDF via `pdfminer`, DOCX via `python-docx`, TXT directly).
2. Chunk text into overlapping windows of 512 tokens with 64-token overlap.
3. Compute dense embeddings $\mathbf{e} \in \mathbb{R}^{768}$ using a sentence-transformer model running locally.
4. Store ($\mathbf{e}$, text chunk, metadata) in Qdrant.

**Retrieval (at inference):**

1. Embed the user query: $\mathbf{q} = f_\theta(\text{query})$.
2. Retrieve the top-$k$ chunks by cosine similarity (Section 4.6.4).
3. Inject retrieved chunks into the LLM prompt as context.

Figure 4.2 summarises both RAG phases as an activity diagram.

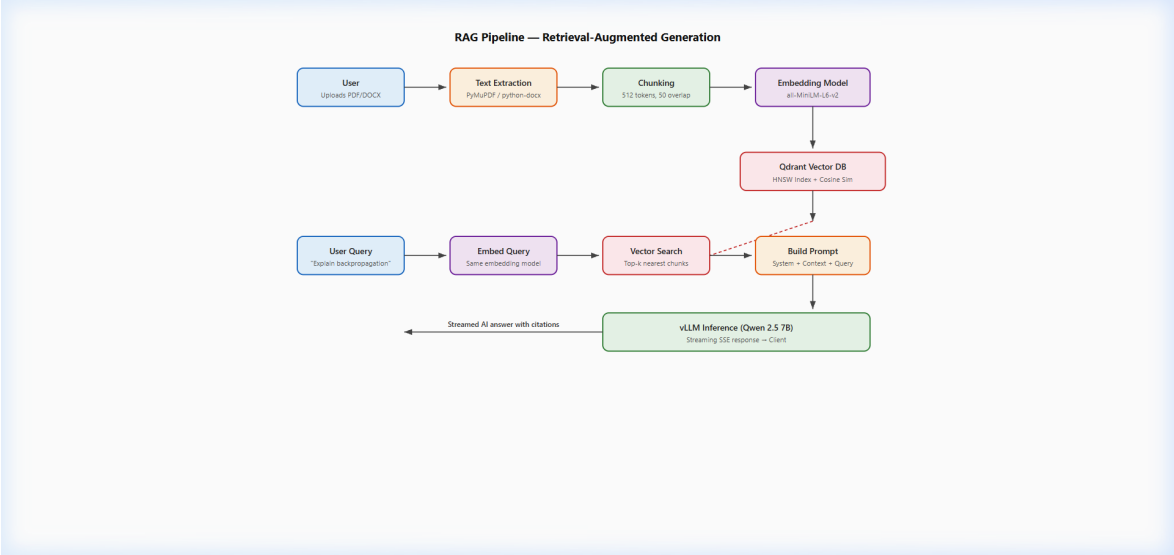


Figure 4.2: RAG Pipeline — Activity Diagram (indexing and retrieval phases)

### 4.6.4 Cosine Similarity and Vector Search

Semantic similarity between a query embedding $\mathbf{q}$ and a document embedding $\mathbf{d}$ is measured by **cosine similarity**:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} = \frac{\sum_{i=1}^{n} q_i d_i}{\sqrt{\sum_{i=1}^{n} q_i^2} \sqrt{\sum_{i=1}^{n} d_i^2}}$$

Values range in $[-1, 1]$; higher values indicate greater semantic relatedness. Qdrant [7] stores all embeddings in HNSW (Hierarchical Navigable Small World) graphs [8], enabling approximate nearest-neighbour search in $O(\log n)$ time rather than the $O(n)$ brute-force scan.

**HNSW search complexity:**

$$T_{\text{search}} = O\left(\log n \cdot \frac{\log n}{\log \log n}\right)$$

For a corpus of $10^6$ documents the search completes in under 5 ms on CPU.

### 4.6.5 Smart Routing Algorithm

MAC's LLM service exposes four model tiers: SPEED, CODE, REASON, and INTELLIGENCE. The routing function selects the optimal tier based on keyword presence in the query:

$$\text{tier}(q) = \begin{cases} \text{CODE} & \text{if } q \cap \mathcal{K}_{\text{code}} \neq \emptyset \\ \text{REASON} & \text{if } q \cap \mathcal{K}_{\text{math}} \neq \emptyset \\ \text{INTELLIGENCE} & \text{if } |q| > \tau_{\text{long}} \\ \text{SPEED} & \text{otherwise} \end{cases}$$

where $\mathcal{K}_{\text{code}}$ is the set of code-related keywords (`code`, `python`, `function`, ...), $\mathcal{K}_{\text{math}}$ is the set of maths-related keywords (`solve`, `prove`, `calculate`, ...), and $\tau_{\text{long}}$ is a token-count threshold. The load balancer then selects the least-loaded available node serving that tier using round-robin scheduling.

### 4.6.6 Voice Pipeline — Whisper STT and Veena TTS

**Speech-to-Text** is handled by OpenAI Whisper [9], a sequence-to-sequence transformer. Given an audio waveform sampled at 16 kHz, Whisper computes an 80-channel log-mel spectrogram and feeds it through an encoder-decoder architecture to produce text. MAC supports 19 Indian languages through Whisper's multilingual checkpoint.

**Text-to-Speech** uses the Veena voice (Piper TTS [10]), trained specifically for Indian-accented English and Hindi. Piper synthesises waveforms using a flow-based acoustic model:

$$p(\mathbf{x}) = p(\mathbf{z}) \left| \det \frac{\partial f^{-1}}{\partial \mathbf{x}} \right|$$

where $f$ is the normalising flow that maps latent Gaussian noise $\mathbf{z}$ to speech waveform $\mathbf{x}$.

## 4.7 Database Design

### 4.7.1 Entity–Relationship Overview

MAC uses **PostgreSQL 16** with 19 ORM models managed by SQLAlchemy 2.0 and Alembic migrations. The core entity hierarchy is:

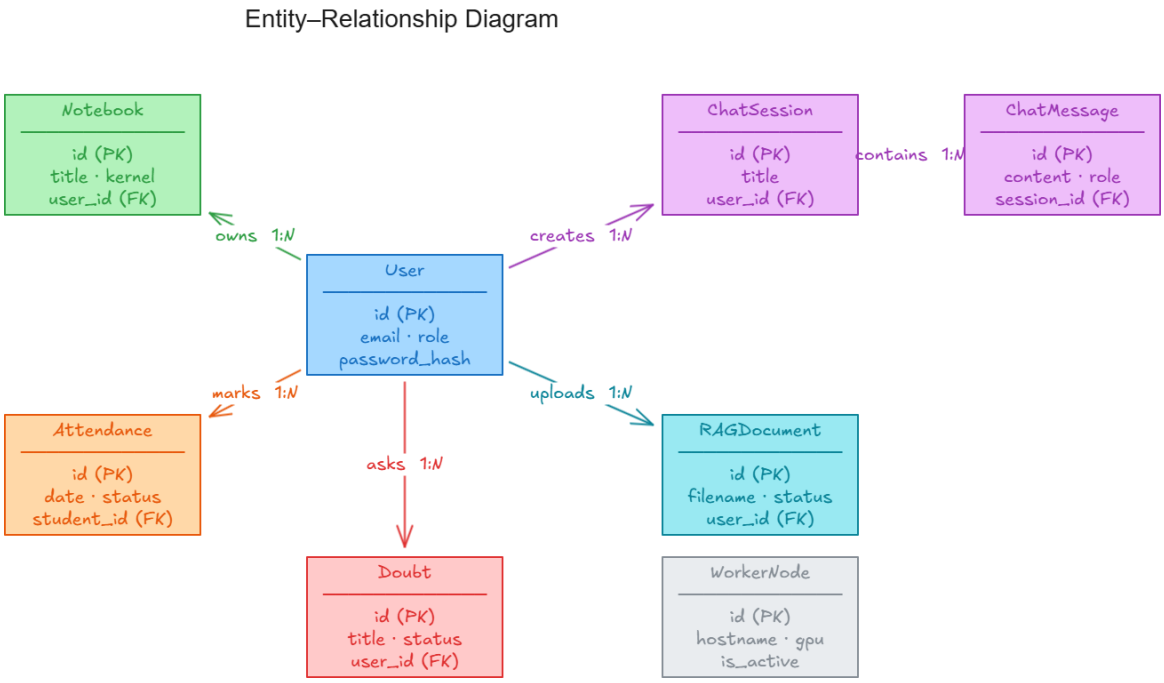


Figure 4.3: Entity–Relationship Diagram — core MAC database entities

Table 4.1: Primary Database Entities

| Entity | Table | Purpose |
|---|---|---|
| User | `users` | Auth, role, password hash |
| APIKey | `api_keys` | Scoped keys, SHA-256 stored |
| Notebook | `notebooks` | Notebook documents |
| NotebookCell | `notebook_cells` | Code/markdown cells |
| RAGCollection | `rag_collections` | Vector DB collection per user |
| Doubt | `doubts` | Q&A forum entries |
| Notification | `notifications` | In-app notifications |
| FeatureFlag | `feature_flags` | Feature enable/disable flags |
| UsageLog | `usage_logs` | Token usage quota tracking |

### 4.7.2 Async Database Access

All database I/O is fully asynchronous using `asyncpg` as the driver and SQLAlchemy's async session factory:

$$\text{Concurrency} \approx \frac{N_{\text{connections}}}{T_{\text{avg query}}}$$

With a pool of 20 connections and average query time of 5 ms, the system can handle approximately 4,000 queries per second — sufficient for hundreds of concurrent users.

### 4.7.3 Schema Design Decisions

- **UUID primary keys** are used throughout to avoid sequential ID enumeration attacks and to enable future distributed sharding.
- **UTC timestamps** on every row (`created_at`, `updated_at`) allow correct ordering regardless of server timezone.
- **Soft deletes** are not used. Rows are hard-deleted for simplicity; an audit log table (`audit_logs`) records significant actions separately.
- Qdrant (vector store) is kept separate from PostgreSQL deliberately: vector similarity queries require specialised index structures (HNSW) not available in standard relational databases.

## 4.8 Frontend Design

### 4.8.1 Vanilla JavaScript SPA Architecture

The MAC frontend is a **Single-Page Application (SPA)** written in plain Vanilla JavaScript — no React, no Vue, no Angular, no TypeScript compilation, no build step of any kind. This was a deliberate architectural decision with concrete benefits:

- **No build toolchain.** A developer changes a `.js` file and refreshes the browser. Zero configuration, zero dependency conflicts.
- **Instant deployment.** Nginx serves static files directly from the filesystem. No `npm install`, no bundler, no CI pipeline for the frontend.
- **Small footprint.** The entire SPA is 15 JavaScript modules totalling approximately 250 KB of application logic (excluding bundled offline libraries).

The router is a hash-based client-side router in `core.js`:

$$\texttt{window.location.hash} \rightarrow \text{render(view)}$$

Navigation triggers re-render of the `#app` container without a full page reload.

### 4.8.2 Progressive Web App (PWA) and Offline Support

MAC is fully PWA-compliant. The Service Worker (`sw.js`) intercepts network requests and serves cached assets when offline:

- Shell HTML, CSS, and all JavaScript modules are pre-cached on install.
- Offline fallback ensures the UI loads even without network access.
- A Web App Manifest enables **Add to Home Screen** on Android and iOS, and **Install App** in Chrome/Edge on desktop.

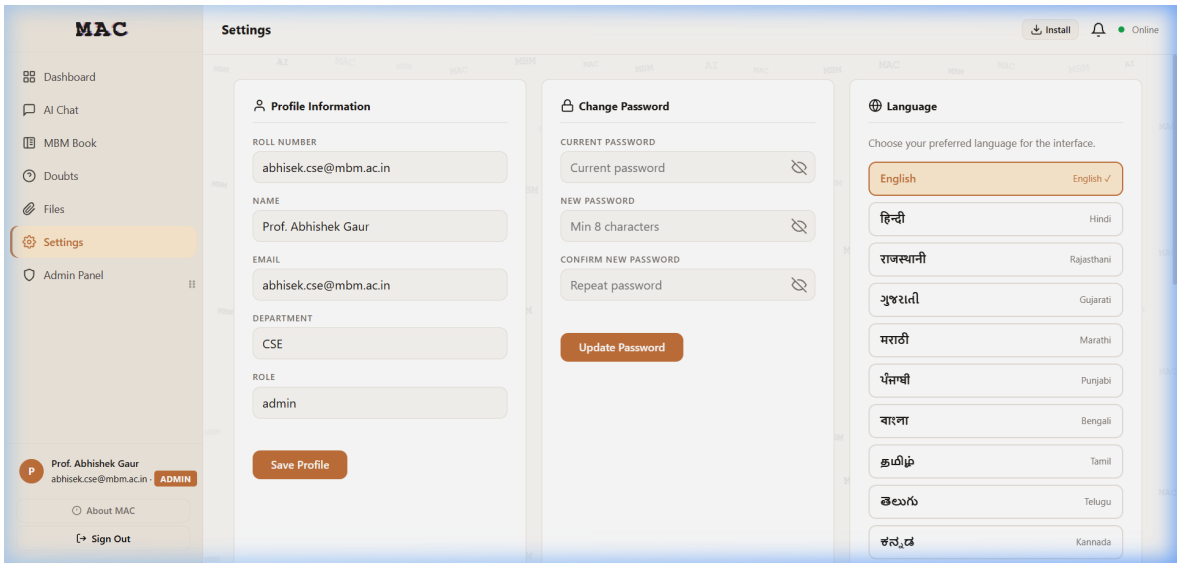


Figure 4.4: MAC Settings and Profile Page

## 4.9 19-Language Offline i18n

All UI strings are stored in a JavaScript translation dictionary in `i18n.js`. Languages supported: English, Hindi, Rajasthani, Gujarati, Marathi, Punjabi, Bengali, Tamil, Telugu, Kannada, Malayalam, Odia, Assamese, Urdu, Nepali, Sinhala, Konkani, Maithili, and Bhojpuri. Language switching is instant (no server round-trip) because all translation data is bundled with the application.

### 4.9.1 Bundled Offline Libraries

To ensure the platform works with no CDN dependencies, the following libraries are shipped as local files under `frontend/libs/`:

Table 4.2: Bundled Frontend Libraries

| **Library** | **Version** | **Purpose** |
|---|---|---|
| Monaco Editor | 0.52.2 | VS Code-quality code editor in notebooks |
| Chart.js | 4.x | Data visualisation (usage graphs, analytics) |
| Highlight.js | 11.x | Syntax highlighting in chat code blocks |
| Mermaid | 10.x | Diagram rendering in notebooks and chat |
| xterm.js | 5.x | Terminal emulator for the web shell |

# Chapter 5

# Features and Modules

## 5.1 AI Chat with Streaming

The chat interface (`chat.js`) communicates with `/query/chat` using **Server-Sent Events (SSE)**. The server streams tokens as they are generated by the LLM, and the browser appends each token to the display in real time. This produces the characteristic *typewriter effect* and reduces perceived latency.

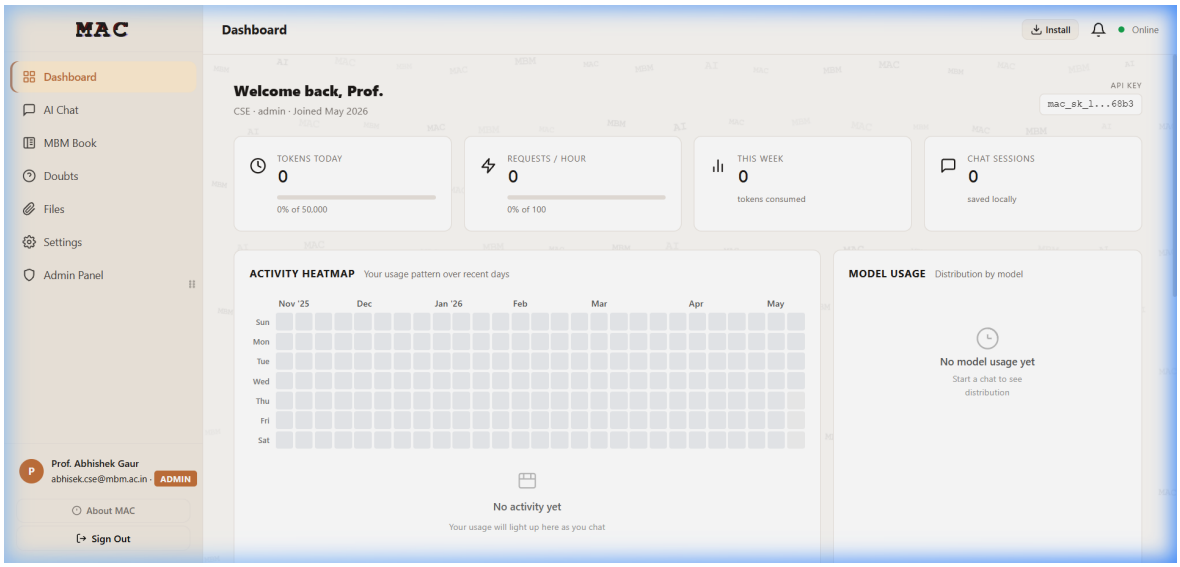


Figure 5.1: MAC Dashboard — System Overview

The chat pipeline supports:

- Multiple named sessions (conversation history persisted server-side).
- **File context injection:** User uploads a PDF/DOCX; the system runs RAG over it and prepends the most relevant chunks to the prompt.
- **Web search integration:** Optionally queries SearXNG and injects search results as context.
- **Model selection:** Users choose the model tier (Speed, Code, Reason, Intelligence) depending on their task.

## 5.2 Computational Notebooks (MBM Book)

MAC implements a Jupyter-compatible notebook system. Each notebook contains ordered cells (code or markdown). The **Kernel Manager** (`kernel_manager.py`) launches Docker containers on demand, one per kernel session, with supported languages: Python 3, JavaScript, Bash, SQL, Java, C++17, and Rust.

Cell execution follows a request-response pattern over WebSocket:

$$\text{Client} \xrightarrow{\texttt{execute\_cell}} \text{KernelManager} \xrightarrow{\texttt{stdin}} \text{Container} \xrightarrow{\texttt{stdout}} \text{Client}$$

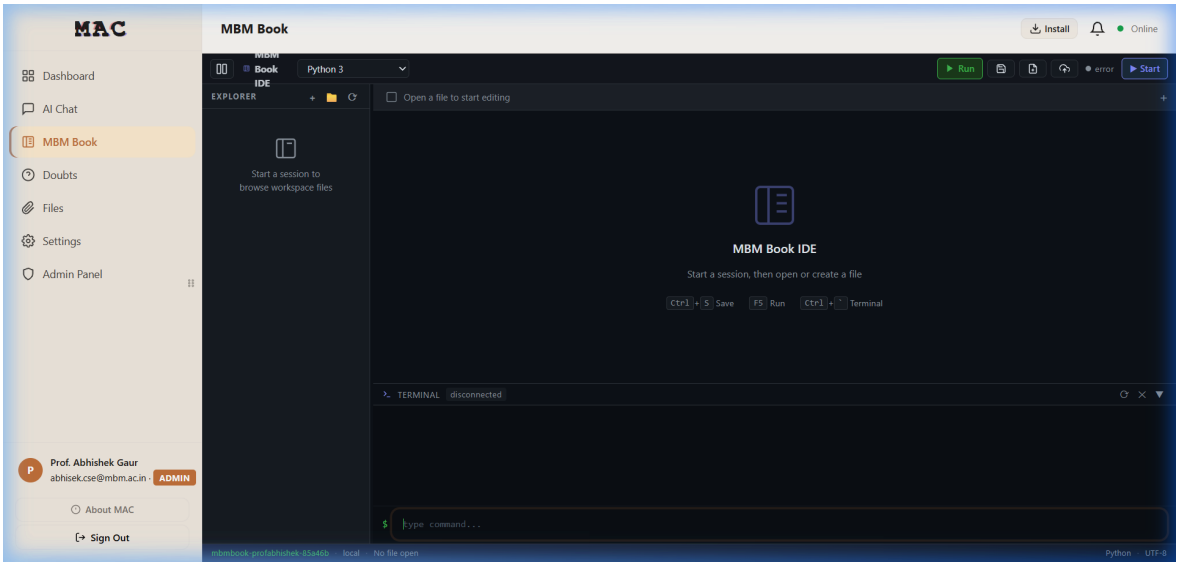


Figure 5.2: MBM Book — Computational Notebook Interface

## 5.3 Other Modules

Table 5.1: Summary of Additional Modules

| **Module** | **Description** |
|---|---|
| Doubts Forum | Students post questions; faculty answer. Threaded replies, faculty notifications |
| File Share | Faculty upload documents; students download. Role-restricted access |
| Voice Chat | Whisper STT transcribes audio; response streamed back as Veena TTS audio |
| Admin Panel | Full user CRUD, quota limits, feature flags, cluster node monitoring |
| System Monitor | Real-time hardware stats (CPU, RAM, GPU VRAM, temperature) |
| Explore (Search) | SearXNG-powered web search with LLM-generated summary |

# Chapter 6

# SDLC and Development Journey

## 6.1 Methodology

The project followed an **Incremental Iterative** development model rather than a strict Waterfall or Scrum process. This choice was driven by the nature of the project: requirements evolved as new LLM capabilities were discovered, and the team was concurrently learning the technologies they were building with.

**Plan** → **Design** → **Build** → **Test** → **Review** → *(repeat)*

Each iteration lasted approximately 2–3 weeks and delivered one working vertical feature slice (e.g., "auth + basic chat" in Iteration 1, "RAG pipeline" in Iteration 3).

## 6.2 Development Timeline

Table 6.1: Development Phases

| **Phase** | **Period** | **Deliverables** |
|---|---|---|
| Phase 1 | Jan 1 – Jan 31, 2026 | Architecture decision, tech stack selection, auth system, basic LLM chat |
| Phase 2 | Feb 1 – Feb 28, 2026 | RAG pipeline, notebooks (MBM Book), file share, doubts forum |
| Phase 3 | Mar 1 – Mar 31, 2026 | Voice pipeline, admin panel, multi-node GPU cluster, feature flags |
| Phase 4 | Apr 1 – Apr 30, 2026 | Stabilisation, testing, Docker GHCR publish, report writing |

## 6.3 Tools Used for Development

Table 6.2: Development Tools and AI Assistants

| Tool | How it was used |
|---|---|
| GitHub Copilot | Real-time code completion, boilerplate generation, inline suggestions |
| Claude Code | Architecture discussions, debugging complex async issues, code review |
| VS Code | Primary IDE |
| Docker Desktop | Local container management during development |
| pgAdmin | Database inspection and query testing |
| Postman | API endpoint testing |
| Git / GitHub | Version control, code collaboration |

## 6.4 Key Challenges and Solutions

1. **Docker + GPU on Windows:** NVIDIA Container Toolkit on Windows requires specific WSL2 configuration. Solved by documenting exact driver versions and providing a setup script.
2. **vLLM model quantisation:** 7B models at full precision require 14 GB VRAM, exceeding the RTX 3060's 12 GB. Solved by using AWQ (4-bit) quantisation which reduces memory to 4–5 GB while preserving 95% of model quality.
3. **Async SQLAlchemy migrations:** Alembic does not natively support async engines. Solved by running migrations synchronously via a compatibility shim in `alembic/env.py`.
4. **JWT logout without statefulness:** JWT is stateless by design but logout requires token invalidation. Solved with Redis-based token blacklisting with matching TTL (Section 4.1).
5. **Frontend without a framework:** Managing complex UI state (multiple chat sessions, live streaming, modals) in Vanilla JS required custom event patterns. Solved by adopting a lightweight publish-subscribe event bus in `core.js`.

# Chapter 7

# Deployment and Distribution

## 7.1 Deployment Architecture Diagram

Figure 7.1 shows the 3-PC cluster topology and container layout.

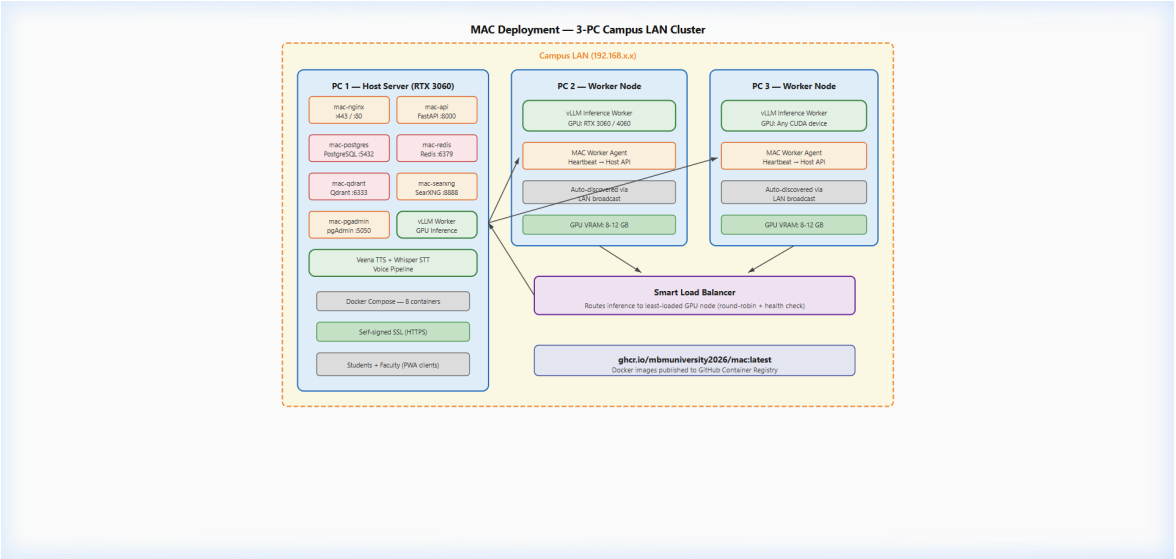


Figure 7.1: MAC Deployment Diagram — 3-PC cluster container layout

## 7.2 Docker Compose Stack

The production deployment uses a single `docker-compose.yml` defining eight services. The entire platform starts with one command: `docker compose up -d`.

Table 7.1: Docker Services

| Service | Image | Function |
|---|---|---|
| `mac-api` | Custom Python image | FastAPI application server |
| `mac-nginx` | nginx:alpine | Reverse proxy, TLS, static files |
| `mac-postgres` | postgres:16 | Primary relational database |
| `mac-redis` | redis:7-alpine | Cache and token blacklist |
| `mac-qdrant` | qdrant/qdrant | Vector database |
| `mac-searxng` | searxng/searxng | Self-hosted web search |
| `mac-pgadmin` | dpage/pgadmin4 | Database administration UI |

A self-signed SSL certificate is generated automatically at first start using the `cryptography` Python library — no `openssl` binary required.

## 7.3 GitHub Container Registry (GHCR)

Container images are built and pushed to `ghcr.io/mbmuniversity2026/mac`. Users can pull the latest image directly:

```
docker pull ghcr.io/mbmuniversity2026/mac:latest
```

## 7.4 Windows Installer

A Windows GUI installer is being developed using **Inno Setup** (`mac-installer.iss`). It provides: hardware detection, role selection (Host / Worker node), automatic firewall rule configuration, SSL certificate generation, and desktop shortcuts. The installer is currently functional but experiencing stability issues in edge cases and remains under active development.

## 7.5 Testing

### 7.5.1 Unit and Integration Tests

The test suite uses **pytest** with `httpx.AsyncClient` for API testing. Tests cover authentication flows, quota enforcement, feature flag evaluation, hardware introspection endpoints, and the key management subsystem.

Table 7.2: Test Coverage by Module

| Test File | Tests | Coverage |
|---|---|---|
| `test_auth.py` | 12 | Login, token refresh, logout, RBAC enforcement |
| `test_keys.py` | 8 | Scoped key creation, revocation, scope checking |
| `test_quota.py` | 6 | Rate limiting, per-user quota tracking |
| `test_features.py` | 5 | Feature flag toggle, per-user override |
| `test_hardware.py` | 4 | Hardware stats endpoint |
| `test_query.py` | 7 | Smart routing logic, streaming endpoint |
| `test_rag.py` | 5 | Collection create, document upload, retrieval |
| `test_search.py` | 3 | SearXNG integration |

### 7.5.2 Manual Testing

All UI flows were tested manually across Chrome (desktop), Firefox (desktop), and Chrome (Android). PWA installation was verified on Android 14 and Windows 11.

## 7.6 Limitations and Future Work

### 7.6.1 Current Limitations

- **Single-GPU bottleneck:** All inference is currently served from one RTX 3060. Under very high concurrent load (>50 simultaneous streaming requests), queue latency increases significantly.
- **Windows installer instability:** The `.exe` installer works in common configurations but crashes in some edge cases (non-standard Windows paths, older Python versions). Under active investigation.
- **No model fine-tuning:** The platform only performs inference. Adapting models to MBM-specific course material requires fine-tuning, which is not yet supported.
- **Limited video studio:** The video editing module is scaffolded but not feature-complete.

### 7.6.2 Future Work

- **LoRA fine-tuning interface:** Allow faculty to upload course material and fine-tune a small LoRA adapter, making the model answer questions about MBM-specific content with higher accuracy.
- **Exam mode:** A locked-down, proctored environment for AI-assisted examinations with logging and guardrails.
- **Federated learning:** Student interaction data (anonymised) could incrementally improve the model without centralising raw data.
- **Stable installer and auto-update:** Complete the Windows installer and add an OTA update mechanism.

## 7.7 Conclusion

This project demonstrates that a production-quality, multi-user AI platform can be built and deployed by a small student team on commodity hardware, at zero ongoing cost, entirely within an institutional network. MAC is not a proof of concept — it is a working system, used on campus, with a real user base.

The platform integrates non-trivial engineering: asynchronous database access, cryptographically secure authentication, LLM inference serving, vector-based semantic search, real-time streaming, Jupyter-compatible kernel execution, multilingual support, and Docker-based cluster orchestration.

We genuinely tried our best to build something useful and to build it right. There are rough edges — the installer, the video module, performance under extreme load — but the core platform is solid and reflects eight months of focused effort, continuous iteration, and a sincere commitment to making AI accessible to every student at MBM University without cost or privacy compromise.

## 7.8 Key API Endpoints

Table 7.3: Selected API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | `/auth/login` | Authenticate and receive JWT |
| POST | `/auth/logout` | Blacklist current token |
| POST | `/query/chat` | LLM chat (streaming SSE) |
| POST | `/rag/collections` | Create RAG collection |
| POST | `/rag/upload` | Upload document for indexing |
| GET | `/notebooks/list` | List user notebooks |
| POST | `/kernels/launch` | Launch a kernel session |
| GET | `/cluster/nodes` | List registered worker nodes |
| GET | `/features/status` | Current feature flag state |

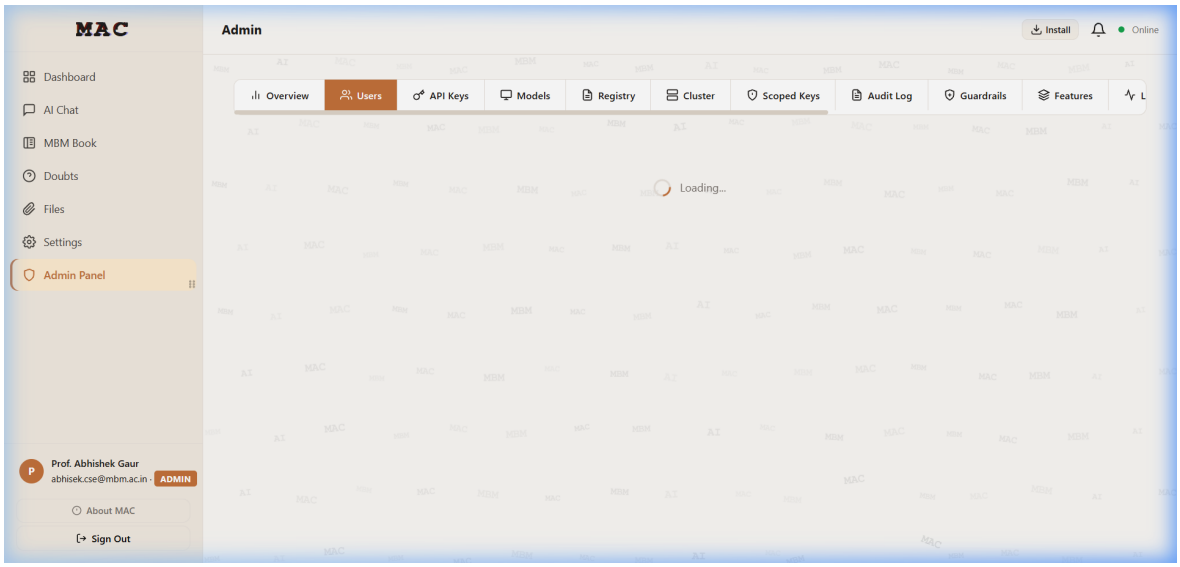


Figure 7.2: MAC Admin Panel — User Management and Cluster Monitoring


Figure 7.3: MAC Doubts Forum — Student Q&A Interface


Figure 7.4: MAC File Sharing Module

# References


[1] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] E. Kasneci et al., "ChatGPT for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102 274, 2023. [Online]. Available: https://doi.org/10.1016/j.lindif.2023.102274

[3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[5] J. Fang, H. Sips, L. Zhang, C. Xu, Y. Zhu, and C. de Laat, "Test-bed implementation of energy and performance of distributed computing," *Future Generation Computer Systems*, vol. 117, pp. 294–307, 2021. [Online]. Available: https://doi.org/10.1016/j.future.2020.11.028

[6] M. Jones, J. Bradley, and N. Sakimura, *JSON web token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519

[7] Qdrant Team, *Qdrant: Vector database for the next generation of AI applications*, Online, 2023. [Online]. Available: https://qdrant.tech/documentation/

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: https://arxiv.org/abs/1603.09320

[9] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: https://arxiv.org/abs/2212.04356

[10] Rhasspy Project, *Piper: A fast, local neural text-to-speech system*, GitHub Repository, 2023. [Online]. Available: https://github.com/rhasspy/piper